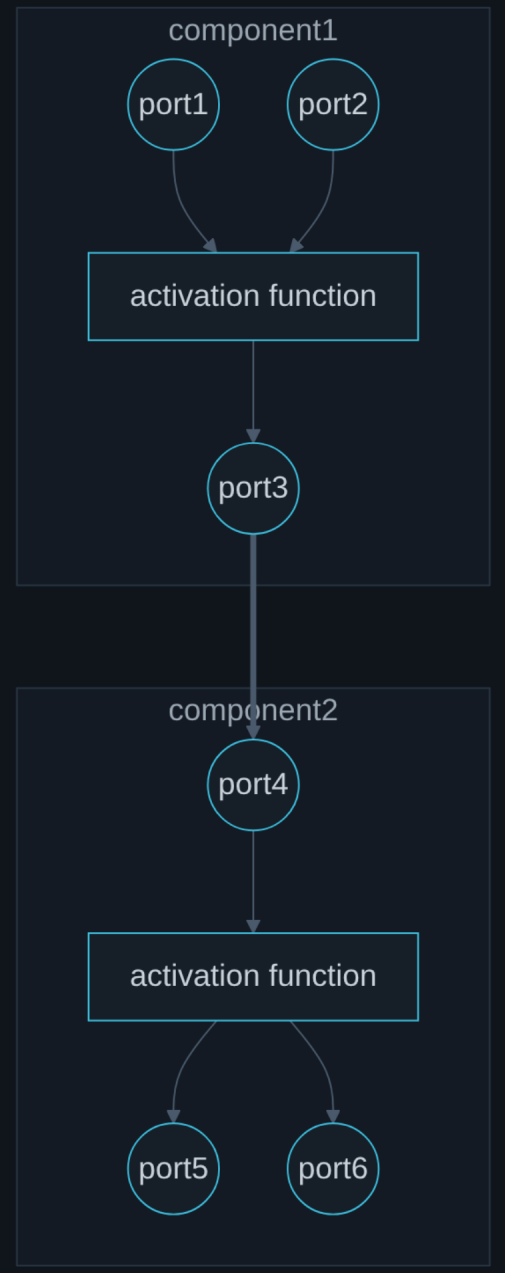
ESM 2026 · Metz, France

Simulation Modeling with FMesh

Applying Flow-Based Programming concepts to build a simulation



Ovsep Avakian

SumUp · Sofia, Bulgaria

avakyan.hovsep@yandex.com

Traditional simulation software vs headless

TRADITIONAL

Specialized suites

AnyLogic | MATLAB | Simulia | Ansys

Rich UI, often CAD. Drag, drop, visualize.

Many paradigms. DES, system dynamics, fluid dynamics, 3D collisions.

Physical primitives. Queues, machines, warehouses, shelves.

HEADLESS

Your own program

No UI.

Abstract, digital-native domains. Software engineering, algorithm and protocol testing, hardware-in-the-loop.

Your own primitives. Whatever your domain is made of.

Simulation tools in general-purpose languages

Python

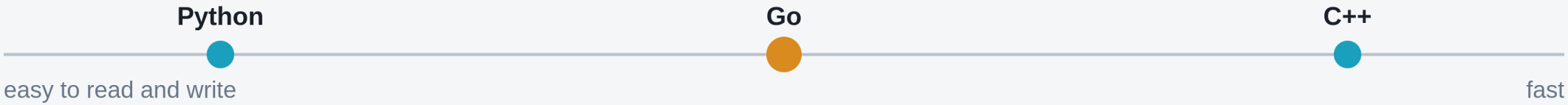
`SimPy · Salabim · Mesa`

C++

`Project Chrono · Jolt Physics · OpenFOAM`

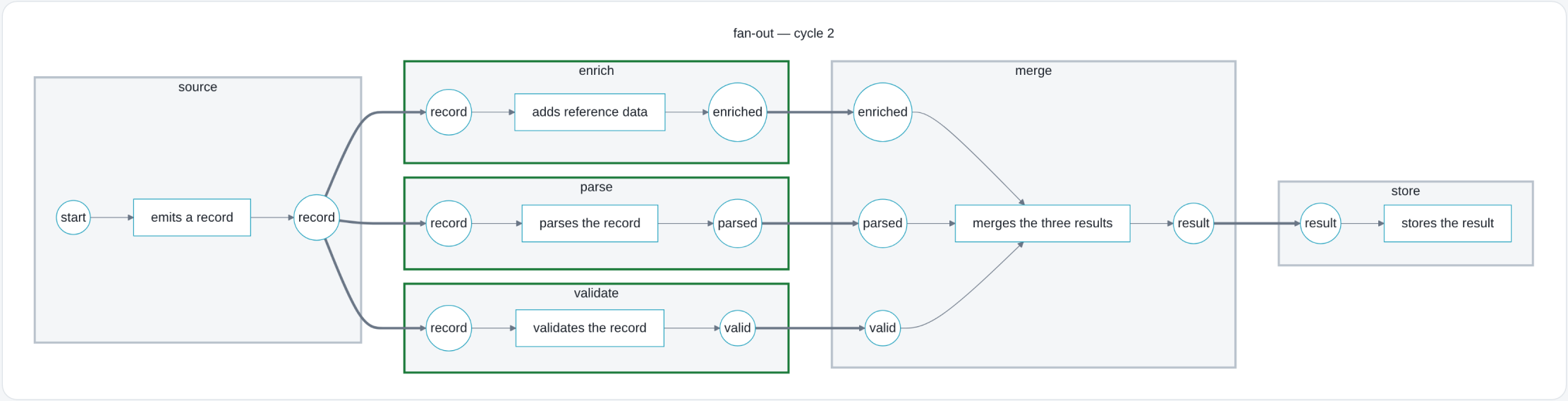
Go

Very few basic libraries, mostly unmaintained



Go can sit between the two. The tooling just isn't there yet.

FMesh: functional mesh



Reactive

A component activates only when a signal arrives on an input.

Concurrent by topology

Ready components run in parallel. No goroutines, channels or locks in your code.

One job

Read inputs, compute, write outputs. A program is a directed graph, cycles allowed.

Flow-Based Programming

1940s
von Neumann
flow diagrams

1962
Petri
Petri nets

1969
J. Paul Morrison
AMPS, the first FBP system

today
agentic AI,
orchestration

Classic FBP

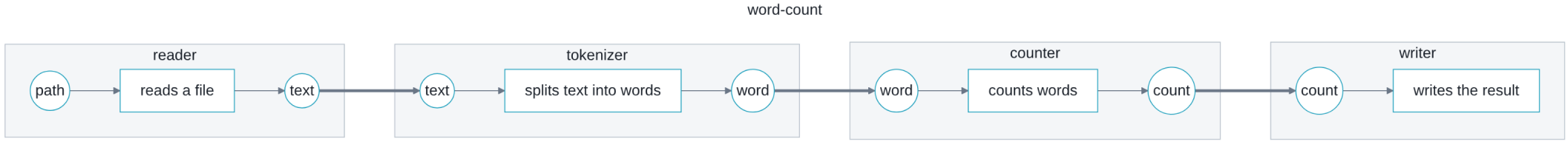
Process: runs all the time
Information Packet
Ports and connections
Black-box components

FMesh, FBP-inspired

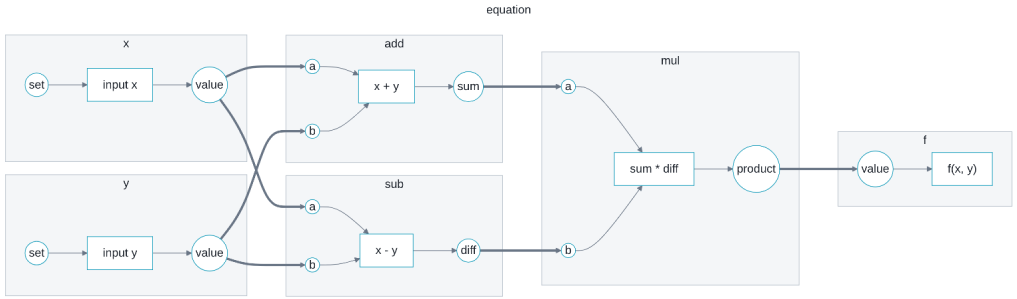
Component: activates on a signal, idle otherwise
Signal
Ports and pipes
Black-box components

The universal data structure

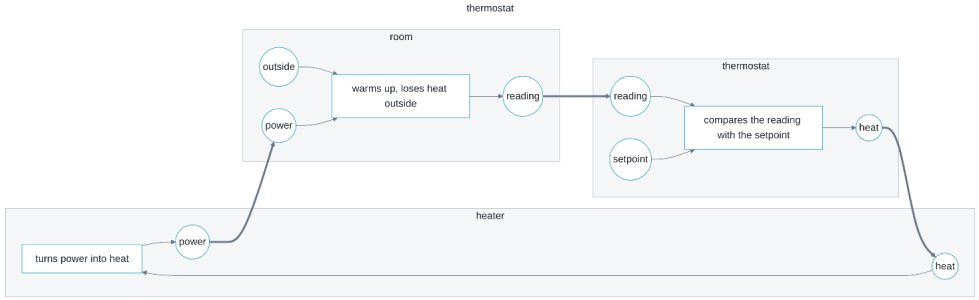
An application



A computation f = (x + y) · (x − y)



A system a thermostat loop



Four steps to a simulation

1

Components

behavior

The entities you care about. Testable in isolation.

2

Connections

topology

How the information flows.

3

Run

execution

Seed the inputs, run the mesh. Repeat in a loop.

4

Analyze

what it means

Interpret the outcomes. Make a business decision.

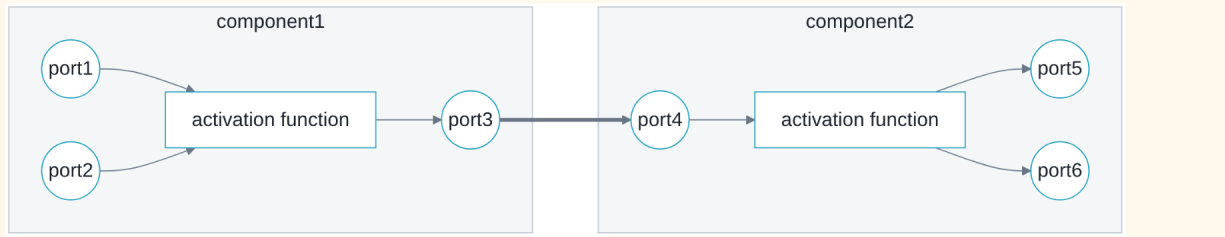
INPUT

Seed signals

parameters, scenario, random seed

RUN LOOP

fm.Run(ctx) once per iteration



OUTPUT

Results

output signals, state, run report

Let's build a restaurant simulation

Components

clock

One tick is one minute.
We activate the clock and the clock activates all other components (time-step simulation).

arrivals

Generates parties of 1–6 guests, more at lunch and dinner.

waiting-list

Parties wait to be seated.

host

Decides who gets which free table.

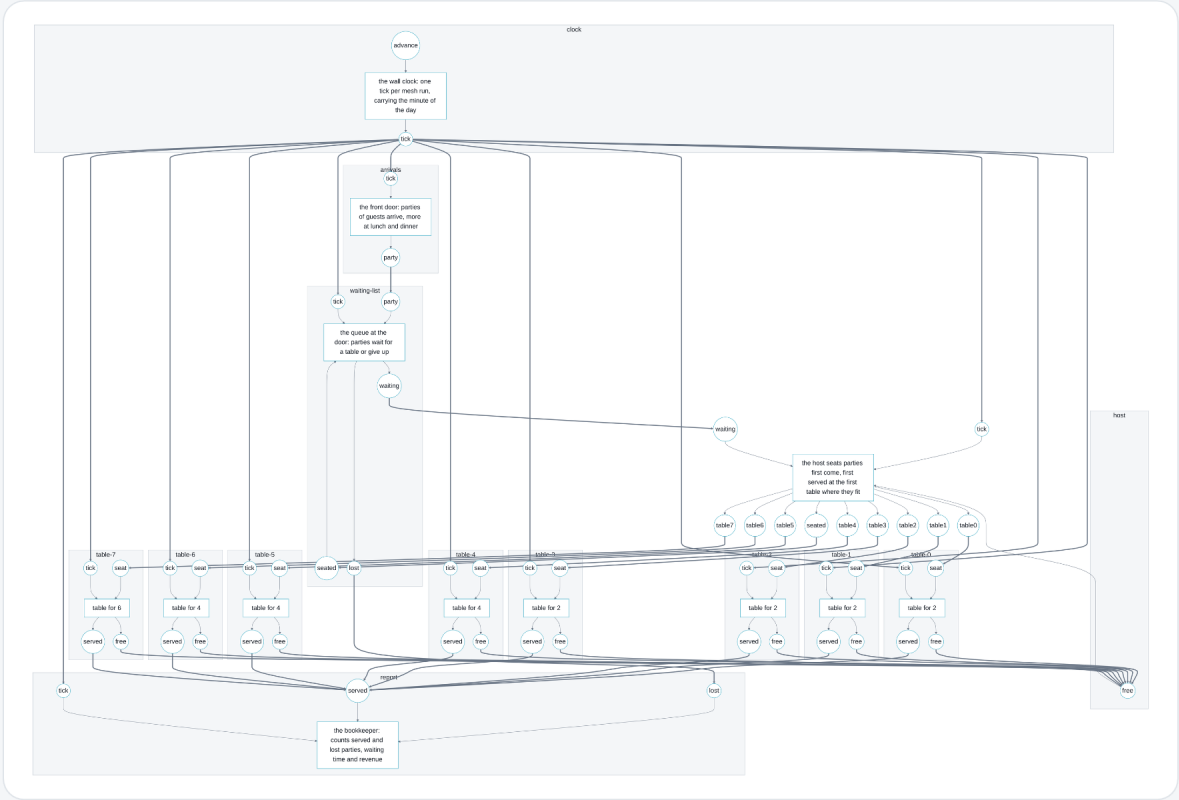
table-0 ... table-7

One component per table.

report

Collects stats like parties and guests served and lost, average wait, utilization, revenue.

Wiring




```
clock.tick              → <every component>.tick
arrivals.party          → waiting-list.party
waiting-list.waiting    → host.waiting
waiting-list.lost       → report.lost
host.seated             → waiting-list.seated
host.table<i>           → table-<i>.seat
table-<i>.free          → host.free
table-<i>.served        → report.served
```

One run is a few cycles

CYCLE 1

The clock ticks

Clock says the minute.
Nobody else has input.

CYCLE 2

Everyone hears the tick

All at once.
Guests arrive.
Tables count the meal down.
Free tables say so.
Queue shows who waits.
Report counts the minute.

CYCLE 3

The host seats, the bookkeeper counts

Host matches parties to tables.
Report books the visits.
Queue admits arrivals.

CYCLE 4

The table takes the party

Tables start the meals.
Queue forgets the seated.
Nothing left to deliver.
Run ends.

Run: one run is one minute

```go
// The clock is a component like any other.
func newClock() (*component.Component, error) {
    return component.New("clock",
        component.WithInputs("advance"),
        component.WithOutputs("tick"),
        component.WithInitialState(func(state component.State) {
            state.Set("minute", 0)
        }),
        component.WithActivationFunc(func(
            _ context.Context, this *component.Component,
        ) error {
            minute, err := this.State().GetTyped[int]("minute")
            if err ≠ nil {
                return err
            }
            if err := this.OutputByName("tick").PutPayloads(minute); err ≠ nil {
                return err
            }
            this.State().Set("minute", minute+1)
            return nil
        }),
    )
}
```

```go
// mesh.go: before every run, nudge the clock.
fm.SetupHooks(func(hooks *fmesh.Hooks) {
    hooks.BeforeRun(func(_ context.Context, _ *fmesh.FMesh) error {
        return clock.InputByName("advance").PutPayloads(true)
    })
})
```

```go
// main.go: a whole day, minute by minute.
for minute := range openingMinutes { // 720 runs
    fm.Run(context.Background())
}
```

Time is a component

FMesh has no clock of its own. Ours is a component with one job: say the minute. The loop just runs the mesh 720 times.

State survives runs

A table keeps its party across runs; the bookkeeper keeps its totals. A day builds up minute by minute.

Analyze: one day, in numbers

guests served

106

of 146 who came

guests lost

40

of 146 · gave up after 20 min

average wait

4.5 min

for the guests served

table utilization

48.3 %

busy ÷ (8 tables × 720 min)

revenue

2 650 EUR

106 guests × 25 EUR

lost revenue

1 000 EUR

40 guests × 25 EUR

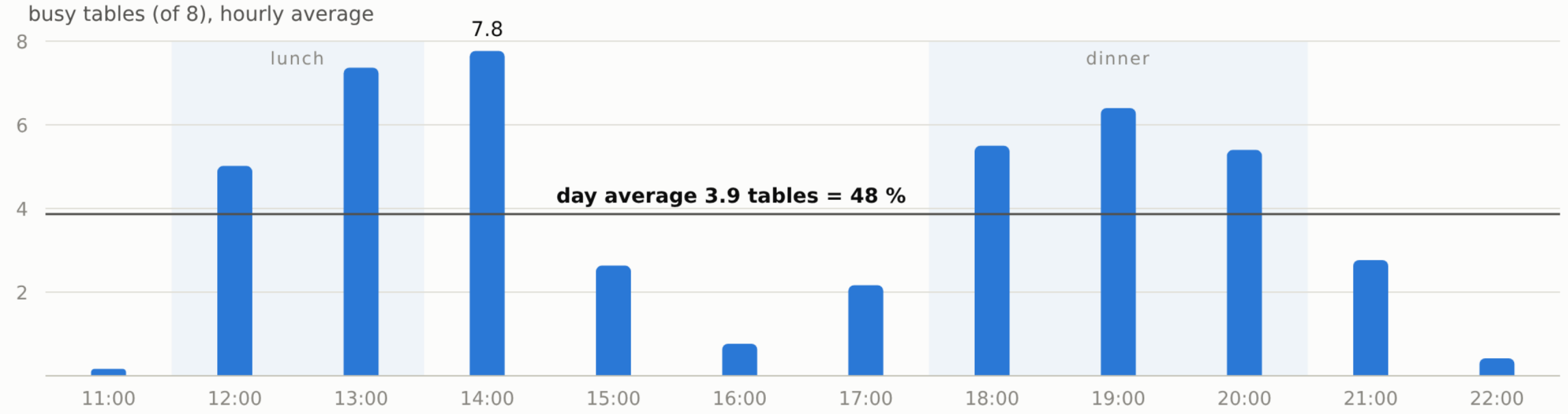
```
=== Restaurant Simulation ===
One mesh run is one minute. Parties arrive, wait for a table, eat and leave;
a host decides who sits where and a bookkeeper counts the day.
Written for the ESM 2026 tutorial on F-Mesh.

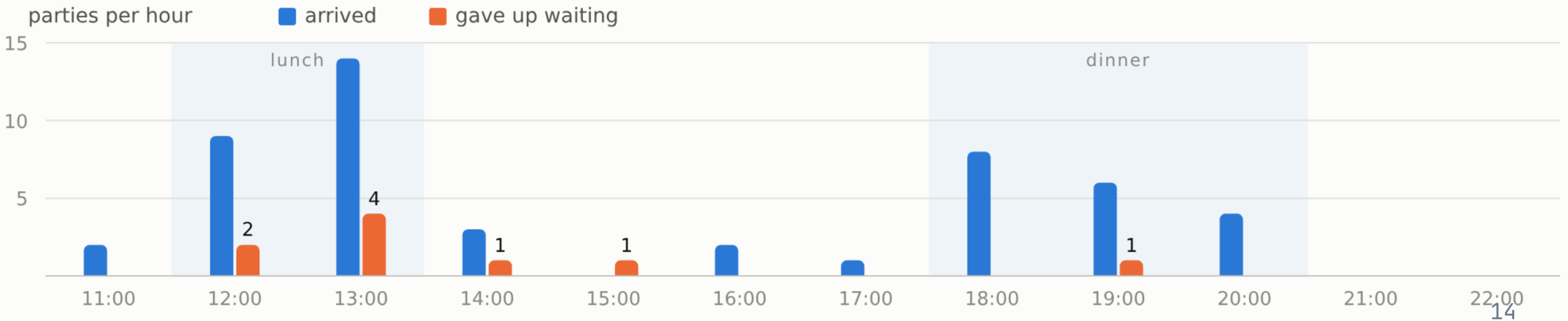
  Host policy    : fifo
  Tables         :    8
  Parties served :   40  (106 guests)
  Parties lost   :    9  (40 guests)
  Average wait   :    4.5 min
  Utilisation    :   48.3 %
  Revenue        : 2650 EUR
  Lost revenue   : 1000 EUR
=== Simulation Complete ===
```

40 of 146 guests walked out. At 25 EUR per guest that is 1 000 EUR of lost revenue a day.

Half empty on average, full at lunch

Nine parties are lost in a day where the tables sit idle half the time

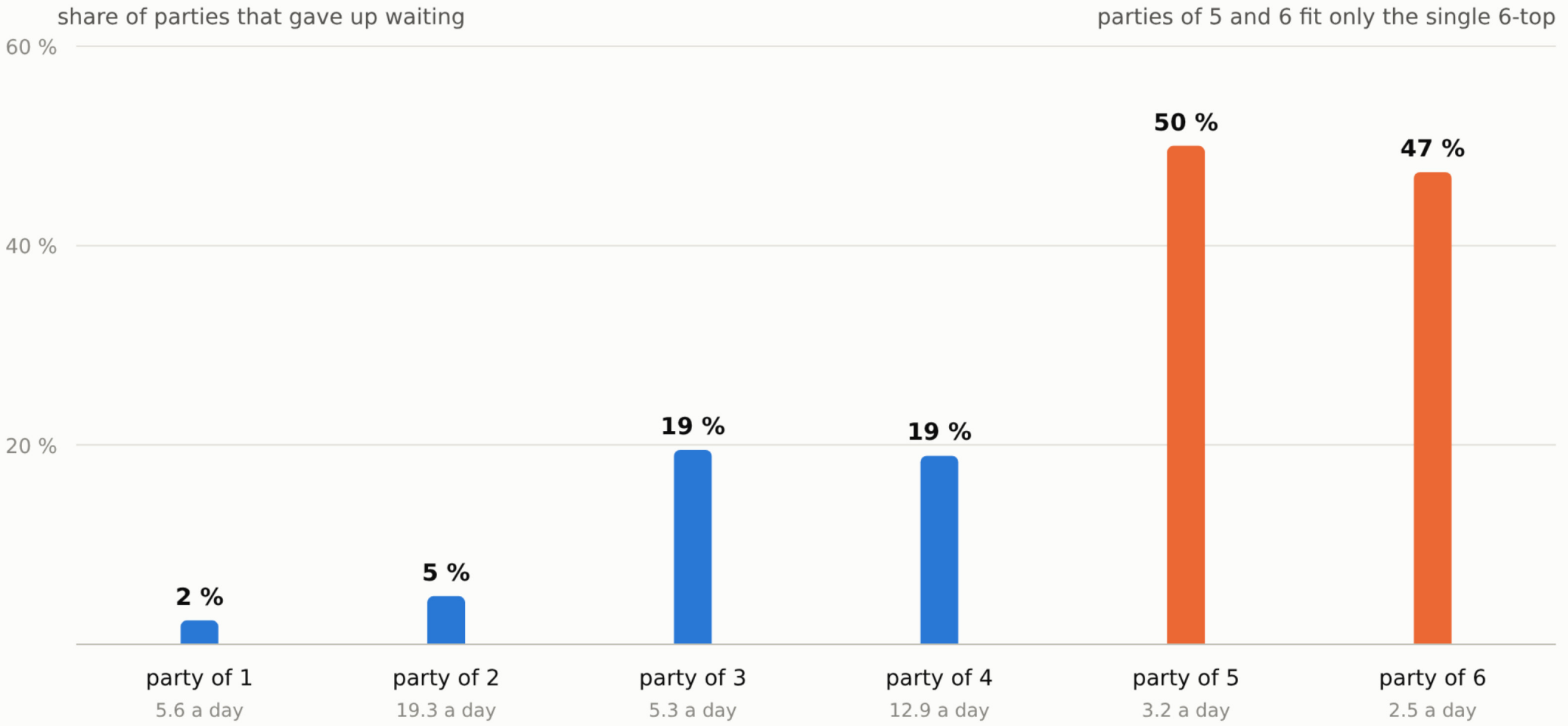




F-Mesh restaurant simulation, one day (seed 1), 8 tables, FIFO host. Lost parties counted in the hour they gave up.

Who walks out: half of the parties of five and six

Couples almost always get a table, big parties wait for the one 6-top and give up



F-Mesh restaurant simulation, 8 tables (four 2-tops, three 4-tops, one 6-top), FIFO host, average of 30 simulated days.

What if we change the seating policy?

```
// Two hosts, same ports, same code; only the policy differs.
func newFIFOHost(numberOfTables int) (*component.Component, error) {
    return newHost("fifo", "…", numberOfTables, chooseFirstFit)
}

func newBestFitHost(numberOfTables int) (*component.Component, error) {
    return newHost("best-fit", "…", numberOfTables, chooseBestFit)
}
```

```
// buildMesh: the whole what-if is this line. Nothing else changes.
host, err := newFIFOHost(len(tableSeats))
host, err := newBestFitHost(len(tableSeats))
```

fifo: first come, first served

Parties in arrival order; each takes the first free table where it fits. A couple may take the 6-top, and the next party of six waits.

best-fit: keep big tables for big parties

Free tables from small to large; each gets the waiting party that leaves the fewest empty seats.

Nothing else changes

The mesh, the tables, the guests, the bookkeeper: untouched. One component is swapped for another with the same ports.

What-if: the same day, the other host

BEFORE · newFIFOHost

```
$ go run .
  Host policy    : fifo
  Tables         :    8
  Parties served :   40  (106 guests)
  Parties lost   :    9  (40 guests)
  Average wait   :    4.5 min
  Utilisation    :   48.3 %
  Revenue        : 2650 EUR
  Lost revenue   : 1000 EUR
```

AFTER THE SWAP · newBestFitHost

```
$ go run .
  Host policy    : best-fit
  Tables         :    8
  Parties served :   41  (111 guests)
  Parties lost   :    8  (35 guests)
  Average wait   :    4.7 min
  Utilisation    :   49.7 %
  Revenue        : 2775 EUR
  Lost revenue   : 875 EUR
```

One party more, five guests more, five guests fewer lost: **+125 EUR**. The wait grew by twelve seconds: a couple now sometimes waits while a party of four takes the free 4-top.

Play with parameters

host	tables	served	lost	avg wait	utilization	revenue EUR	lost EUR
fifo	8	40	9	4.5 min	48.3 %	2 650	1 000
best-fit	8	41	8	4.7 min	49.7 %	2 775	875
fifo	10	46	3	3.7 min	44.3 %	3 300	350
best-fit	10	46	3	3.7 min	44.3 %	3 300	350

ONE DAY

Two more tables: +650 EUR. The smarter host: +125 EUR with eight tables, nothing with ten. Is that the host — or just this day?

Analyze: 30 days, not one

host	tables	guests served / day	guests lost / day	avg wait	utilization	revenue EUR / day	lost EUR / day
fifo	8	112.7	30.0	3.1 min	48.4 %	2 818	751
best-fit	8	114.1	28.8	2.9 min	48.3 %	2 853	721
fifo	10	126.4	16.3	2.2 min	42.3 %	3 161	408
best-fit	10	127.0	15.8	2.1 min	42.3 %	3 175	394

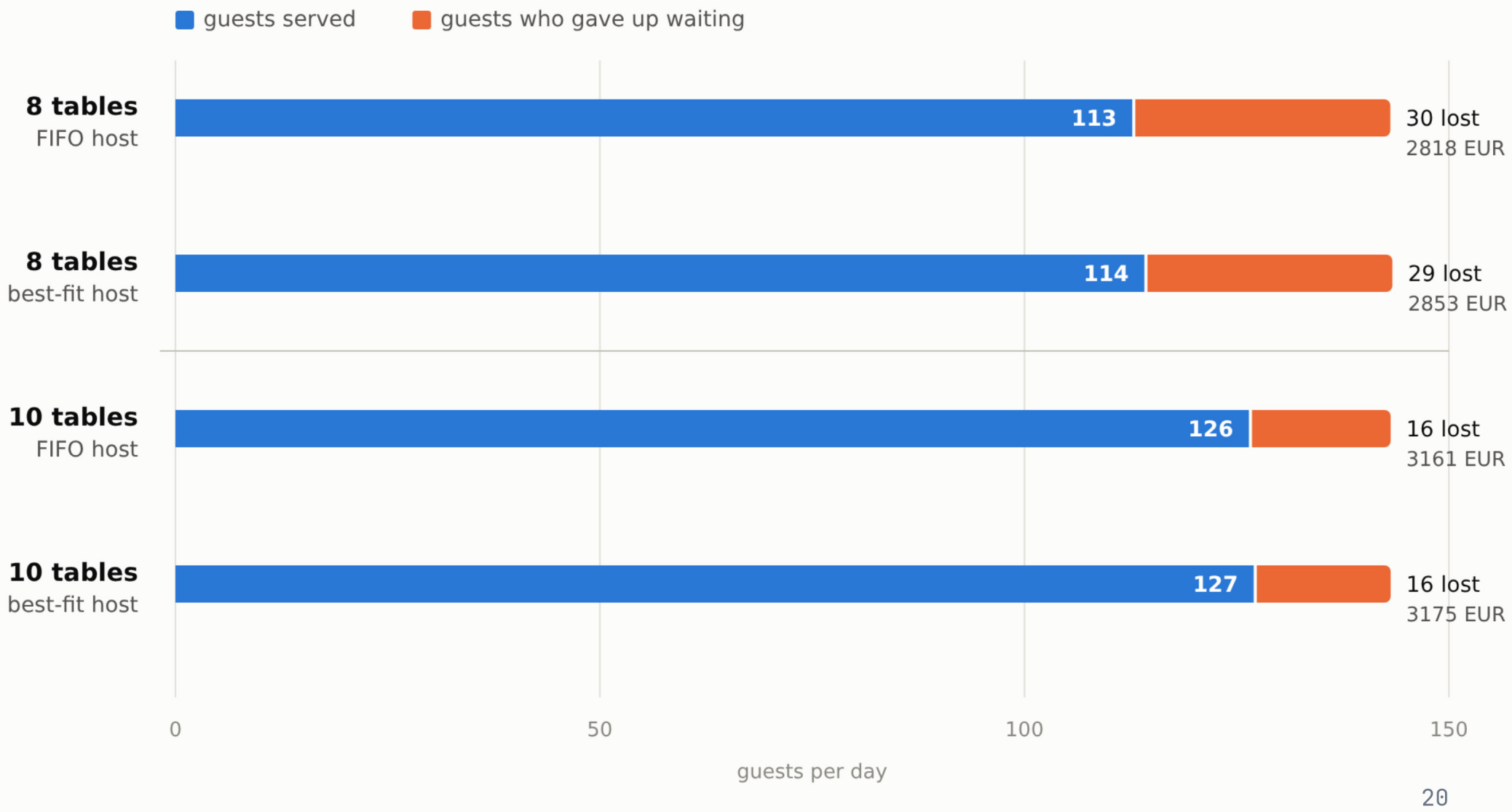
DECISION

Add two 4-tops: about +340 EUR a day, before the cost of the tables. A best-fit host adds 15 to 35 EUR a day: real, but small.

Tables are the big lever: 14 fewer guests lost a day. The host helps most when tables are scarce.

Two extra 4-tops bring 14 more guests a day, the seating policy brings one

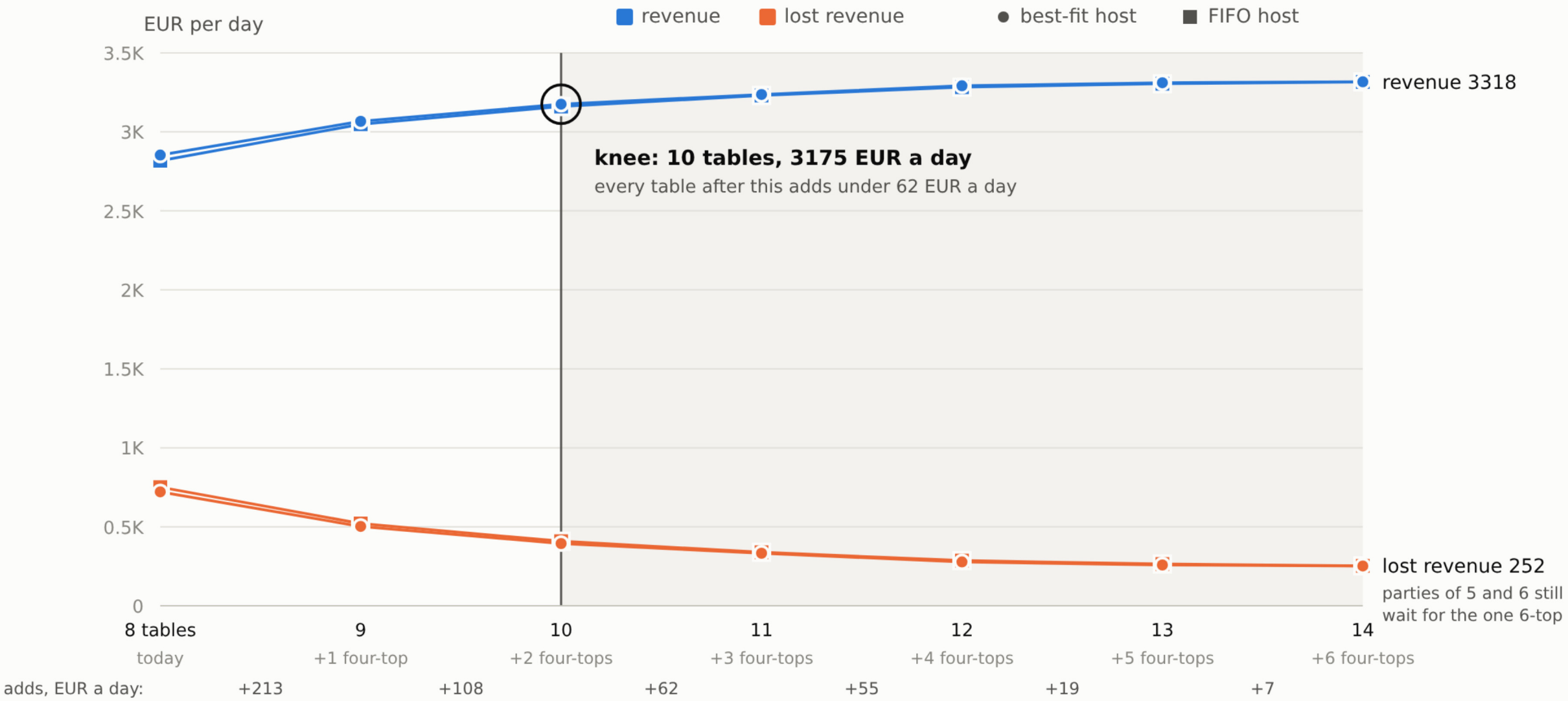
Guests per day under four what-if scenarios, averaged over 30 simulated days



F-Mesh restaurant simulation, average of the same 30 simulated days per scenario, 25 EUR per served guest.

Two more tables are worth it, the sixth is not, and the host hardly matters

Revenue and lost revenue per day as 4-tops are added, for both seating policies



F-Mesh restaurant simulation, 30 simulated days per point, 25 EUR per guest. Tables are added as 4-tops to the plan of four 2-tops, three 4-tops and one 6-top.

Try it yourself

```
git clone https://github.com/hovsep/fmesh-examples
cd fmesh-examples/simulation/restaurant

go run .
```

Other things you can change, one line each

tableSeats	how many tables, and how big
newFIFOHost → newBestFitHost	which host seats the guests
patienceMinutes	how long guests wait before they leave
openingMinutes	how long the restaurant is open
revenuePerGuest	how much one guest pays
seed	a different random day (run many to average)
lunchRate, dinnerRate	how many guests arrive at peak hours

Links

`github.com/hovsep/fmesh`
`github.com/hovsep/fmesh-examples`
`github.com/hovsep/fmesh-export`